Weather ETL Pipeline with Apache Airflow

Data Analyst Interview Preparation Guide

A concept-level walkthrough of every file in the project, Airflow fundamentals for a fresher, the data-analyst angle (SQL, data quality, ETL thinking), honest limitations, and a bank of interview questions with model answers.

Part	What you get
1	Project pitch, resume talking points, project at a glance
2	Apache Airflow fundamentals (everything a fresher needs)
3	File-by-file deep dive of this repository
4	Data-analyst angle: ETL concepts, data quality, SQL on weather_data
5	Design decisions, limitations, and improvements you can discuss
6	Interview question bank (116 questions with answers, deployment questions in Part 8)
7	Behavioural answers, traps to avoid, cheat sheets, last-day checklist
8	Deployment to Astro Cloud with Supabase Postgres: steps, problems solved, deployment questions

How to use this document: Answer in your own words. Interviewers can tell memorised text from understanding. Where a number or claim is not in the code (run counts, row counts, dashboards), do not invent it. Run the pipeline, check the real values, and quote those.

Source of truth: Everything below is derived from the files in the ETLWeather-main folder: dags/etlweather.py, tests/dags/*.py, Dockerfile, airflow_settings.yaml, requirements.txt, packages.txt, .astro/config.yaml, README.md, LICENSE, plus the deployment steps actually run on Astro (Part 8). Items I could not verify from the files (for example the exact Airflow version inside the Runtime image) are marked "verify".

Part 1. The Project in One Page

1.1 The 30-second pitch (memorise the shape, not the words)

"I built an automated ETL pipeline using Apache Airflow that pulls current weather data from the Open-Meteo API, cleans and validates it, and loads it into a PostgreSQL table. It runs daily on a schedule, retries on failure, rejects implausible readings through a data-quality check, and the load is idempotent, so re-running never creates duplicates. The location is configurable at trigger time, defaults to London, and the pipeline is covered by pytest tests. It runs locally in Docker using the Astronomer CLI."

1.2 The 2-minute version: structure it as Problem, Approach, Result, Learning

- **Problem:** weather data is only useful for analysis if it is collected consistently over time. Doing it by hand or with a cron script gives no visibility, no retries, no data checks.
- **Approach:** one DAG with 5 tasks: create_table, extract, transform, validate, load. Airflow provides scheduling, retries, logging, a UI, and dependency management.
- **Engineering choices:** secrets live in Airflow Connections (not in code); a UNIQUE constraint with ON CONFLICT DO NOTHING makes the load idempotent; a validation gate stops bad data before it reaches the table; AirflowFailException prevents pointless retries on bad data; Params let a user change location without editing code.
- **Result:** a queryable weather_data table that grows by readings, ready for SQL analysis or a BI dashboard.
- **Learning:** orchestration vs processing, idempotency, event time vs load time, testing DAGs without calling real services.

1.3 Resume bullet ideas (adapt, and only keep what is true)

- Built a daily ETL pipeline in Apache Airflow (TaskFlow API) ingesting REST API data into PostgreSQL, orchestrating 5 tasks with retry and failure handling.
- Implemented idempotent loads using a composite UNIQUE key and INSERT ... ON CONFLICT DO NOTHING, eliminating duplicate rows on reruns.
- Added a data-quality gate (range check on temperature) that fails fast without retries to keep bad records out of the warehouse.
- Managed credentials via Airflow Connections and parameterised the DAG with validated runtime Params (latitude/longitude).
- Wrote 15 pytest tests (12 for the pipeline: structure, dependencies, transform, validation, mocked API/DB calls; 3 template hygiene checks); containerised with Astro CLI and Docker.
- Deployed the DAG to Astronomer (Astro) on AWS with connections as secret environment variables and a Supabase PostgreSQL destination; verified the end-to-end cloud run.

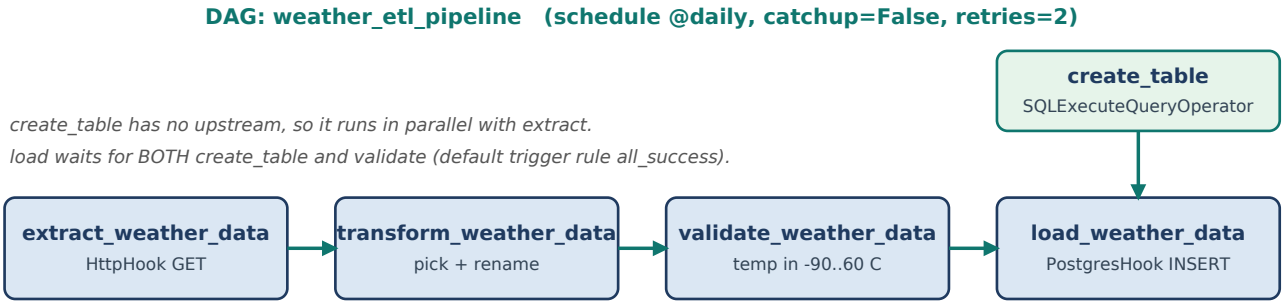
Verify: The 12 pipeline tests come from counting test functions plus the 3 parametrised cases in tests/dags/test_etlweather.py. Run `astro dev pytest` to confirm the number before quoting it. It was run on Runtime 13.12.2 and 15 tests passed in total.

1.4 Project at a glance

Item	Value
DAG id	weather_etl_pipeline
Schedule	@daily (midnight UTC), catchup=False, start_date 2024-01-01
Source	Open-Meteo forecast API, endpoint /v1/forecast with current_weather=true (free, no API key)

Item	Value
Sink	PostgreSQL table weather_data (via connection postgres_default)
Tasks	create_table, extract_weather_data, transform_weather_data, validate_weather_data, load_weather_data
Retries	2 retries, 5 minutes apart (so at most 3 attempts per task)
Runtime	Astro Runtime 13.12.2 (Airflow 2.11.2). Originally 12.1.1, which Astro has retired, so the Dockerfile was upgraded
Deployment	Astro Cloud (AWS us-east-1, dev mode), destination Supabase PostgreSQL via Session pooler (see Part 8)
Tooling	Astronomer CLI (astro dev start), Docker, pytest
Language	Python for the DAG, SQL for the table and insert
Licence	GNU GPL v3 (LICENSE file)

1.5 Flow diagram (be able to draw this on a whiteboard)



Data passed between tasks as XCom (small dicts) - automatic with the TaskFlow API.

Source: Open-Meteo API (JSON) *Sink: weather_data table in PostgreSQL*

1.6 What one run looks like, step by step

- Scheduler notices the @daily interval has ended and creates a DAG run. Task instances are created for 5 tasks.
- create_table and extract_weather_data have no upstream tasks, so both start immediately.
- extract calls GET https://api.open-meteo.com/v1/forecast?latitude=..&longitude=..&current_weather=true using HttpHook, returns the JSON (saved as an XCom).
- transform reads that JSON and returns a flat dict: latitude, longitude, temperature, windspeed, winddirection, weathercode, observed_at.
- validate checks temperature is present and between -90 and 60 C. If not, raises AirflowFailException: the task fails immediately with no retry, and load is marked upstream_failed.
- load runs only after create_table and validate both succeeded; it inserts one row, and ON CONFLICT DO NOTHING silently skips an exact repeat of (latitude, longitude, observed_at).

Part 2. Apache Airflow Fundamentals for a Fresher

You do not need to be an Airflow expert for a data analyst role. You need to explain what it is, why it exists, and the 15 or so vocabulary items below, and connect each one to a line of your own project.

2.1 What is Airflow and why does it exist?

Apache Airflow is an open-source **workflow orchestrator**: you define workflows as Python code, and Airflow schedules them, runs the steps in the right order, retries failures, records history, and shows everything in a web UI. It orchestrates; it does not process big data itself. Think of it as a conductor, not an orchestra: the heavy work happens in your Python code, a database, Spark, dbt, etc.

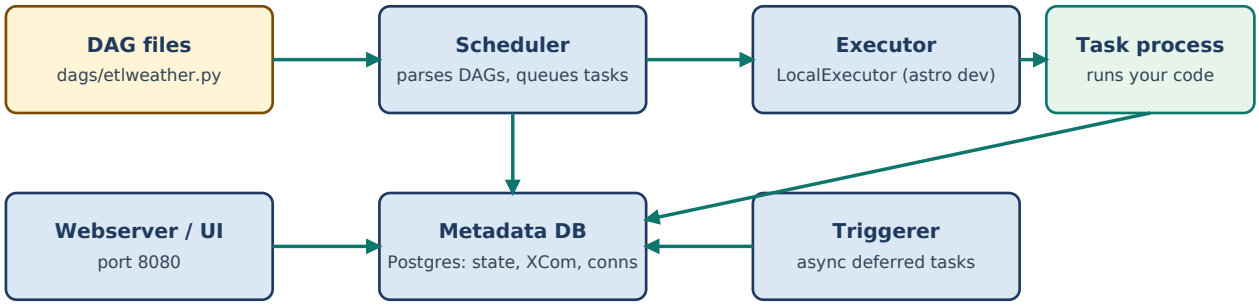
Without Airflow (cron + scripts)	With Airflow
No dependency handling between scripts	Explicit task dependencies (A >> B)
Failures silent unless you build alerts	Retries, failure states, callbacks, email/Slack alerts
No central history or logs	UI with run history, per-task logs, durations
Backfilling is manual	Built-in catchup/backfill by date interval
Secrets scattered in scripts	Connections and Variables (can use a secrets backend)
Hard to scale or monitor	Executors, pools, concurrency limits, metrics

2.2 Core concepts (the vocabulary)

Term	Meaning	In this project
DAG	Directed Acyclic Graph: tasks plus dependencies, no loops. The workflow definition.	weather_etl_pipeline
Task	One unit of work in a DAG.	extract_weather_data, load_weather_data, ...
Task instance	A task for one specific DAG run, with its own state and logs.	extract on a given day's run
DAG run	One execution of the whole DAG for a given logical date / interval.	One daily run or manual trigger
Operator	A pre-built template for a type of task (run SQL, call a bash command, send email...).	SQLExecuteQueryOperator for create_table
TaskFlow API / @task	Decorator style: a normal Python function becomes a task; return values pass as XComs automatically.	extract, transform, validate, load
Hook	Reusable interface to an external system. Wraps connection details and client code. Used inside tasks.	HttpHook, PostgresHook
Connection	Stored credentials/host info for external systems, referenced by conn_id.	open_meteo_api, postgres_default
XCom	Cross-communication: small data passed between tasks, stored in the metadata DB.	dicts returned by extract/transform/validate
Params	Runtime parameters with optional validation, editable when triggering a DAG manually.	latitude, longitude
Schedule	When the DAG runs: cron string, preset (@daily), timedelta, or dataset-based.	@daily
Catchup	Whether to create runs for all missed intervals since start_date.	False

Term	Meaning	In this project
Retries / retry_delay	How many times and how long to wait before re-running a failed task.	2 retries, 5 min
Trigger rule	Condition under which a task runs based on upstream states. Default all_success.	load needs create_table AND validate
Executor	Mechanism that actually runs task instances (Local, Celery, Kubernetes).	Local-style executor in astro dev (verify)
Provider package	Pip package adding operators/hooks for a service (http, postgres, common-sql).	pre-installed in Astro Runtime

2.3 Architecture: the components



Scheduler decides WHAT runs WHEN; executor decides WHERE; task process does the work.
Everyone talks to the metadata DB; it is the single source of truth for run/task state.

Component	Job
Scheduler	Parses DAG files, creates DAG runs on schedule, decides which task instances are ready and hands them to the executor.
Executor / workers	Runs the task code. LocalExecutor runs tasks as processes on the same machine; Celery/Kubernetes executors distribute across machines.
Webserver (UI)	Web UI: Grid, Graph, logs, trigger/clear/pause. In Airflow 3 this role is the API server.
Metadata database	Stores DAG runs, task states, XComs, connections, variables, users. Airflow itself needs this (Postgres/MySQL; SQLite only for toy use).
Triggerer	Runs deferrable tasks asynchronously (waiting on external events without holding a worker slot). The README lists it as one of the 4 local containers.
DAG folder	Where your .py files live; scheduler re-parses them regularly.

This project: README says astro dev start launches 4 containers: Postgres (metadata DB), Webserver, Scheduler, Triggerer. There is no separate worker container, consistent with a local executor setup.

2.4 Scheduling concepts that confuse everyone

- **Data interval:** every run covers a time window. For @daily the run for 2025-01-01 covers 2025-01-01 00:00 to 2025-01-02 00:00 and is triggered *after* that window ends (at 2025-01-02 00:00). Airflow was designed for processing "the data of that interval".
- **Logical date** (formerly execution_date): the START of that interval, not the wall-clock time the run actually started.
- **start_date:** the earliest date from which intervals are considered. Not "when it starts running".
- **catchup=True** would create a run for every missed interval from start_date to now; **catchup=False** creates only the latest one. This project uses False because the API only returns

the *current* weather. Backfilling past days would store today's weather labelled as old runs.

- **Backfill:** manually re-running a DAG over a past date range.
- **Idempotency:** running the same task multiple times with the same input produces the same final state. Critical for retries and backfills. This project achieves it with CREATE TABLE IF NOT EXISTS and ON CONFLICT DO NOTHING.
- **Cron presets:** @daily = 0 0 * * *, @hourly = 0 * * * *, @weekly, @monthly, @once. Timezone default is UTC.

2.5 Task states and what they mean

State	Meaning
none / scheduled / queued	Created, waiting for scheduler/executor/slot
running	Currently executing
success	Finished OK
failed	Failed after exhausting retries (or AirflowFailException)
up_for_retry	Failed but will be retried after retry_delay
upstream_failed	Skipped because an upstream task failed (what load becomes if validate fails)
skipped	Skipped on purpose (branching, short-circuit)

2.6 TaskFlow API vs classic operators

Classic style creates operator objects (PythonOperator(task_id=..., python_callable=...)) and wires XComs manually with xcom_pull. TaskFlow (Airflow 2.0+) lets you decorate a normal function with @task; calling one task's output as another's argument builds the dependency and the XCom passing for you. This project mixes both: SQLExecuteQueryOperator (a classic operator) for DDL, and @task functions for Python logic. Then it uses `create_table >> load_weather_data(validated_data)` to add the extra dependency.

```
# Classic                                   # TaskFlow
def extract(): ...                          @task()
t1 = PythonOperator(task_id='e',            def extract(): ...
        python_callable=extract)            data = extract()
t2 = PythonOperator(... xcom_pull ...)      transform(data)   # dependency + XCom automatic
t1 >> t2
```

2.7 XCom: what and why to be careful

- XCom values are serialised (JSON) and stored in the metadata DB. Good for small things: IDs, file paths, small dicts. Bad for big DataFrames or files.
- Here the payloads are tiny dicts (the API JSON and one flattened row). Fine.
- For bigger data, write to storage (S3, database, file) and pass only the reference through XCom.

2.8 Hooks, Operators, Sensors: which is which?

- **Operator:** does a defined action as a task (e.g. execute SQL). Declarative.
- **Sensor:** a special operator that waits for a condition (file exists, partition arrives, API says ready).
- **Hook:** a connection helper you call from Python code. Operators use hooks internally; you use hooks directly when you write your own @task.
- Rule of thumb: if a ready-made operator exists for what you need, use it; if you need custom logic, use @task plus a hook.

2.9 Connections and Variables

- **Connection:** structured credentials for an external system (conn_id, conn_type, host, login, password, port, schema, extra). Code refers only to the conn_id, so secrets are not in the DAG file.
- **Variable:** a key/value setting for config that changes without redeploying code.
- Both can be set in the UI, via environment variables, or via a secrets backend (Vault, AWS Secrets Manager). In this project, airflow_settings.yaml seeds them for local development only.

2.10 Other things interviewers may probe (short answers)

Topic	One-line answer
Executors	Local (single machine), Celery (distributed workers via queue), Kubernetes (pod per task). Choice depends on scale.
Pools	Limit concurrent tasks hitting a shared resource (e.g. max 5 DB connections).
Sensors	Tasks that wait for a condition; poke vs reschedule vs deferrable mode.
Branching	BranchPythonOperator / @task.branch selects which downstream path runs.
Trigger rules	all_success (default), all_failed, one_failed, all_done, none_failed...
Dynamic task mapping	expand() creates N task instances at runtime from a list (e.g. one per city). README mentions it in the template example.
Datasets (assets in Airflow 3)	Data-aware scheduling: DAG B runs when DAG A updates a dataset.
Jinja templating	{{ ds }}, {{ logical_date }} etc. rendered into operator fields at runtime.
SLA / callbacks	on_failure_callback, on_success_callback, email_on_failure for alerting.
Deferrable operators	Free the worker slot while waiting; the triggerer handles the wait asynchronously.
DAG best practices	Keep top-level code light (no API calls at import time), idempotent tasks, small XComs, retries, meaningful ids/tags.
Airflow 3	Newer major version: API server replaces webserver, task SDK (airflow.sdk), assets rename, logical_date/data-interval changes. This project is on the 2.x line.

2.11 Airflow vs alternatives (for context)

Tool	Positioning
cron	Just time-based triggers. No dependencies, retries, UI, history.
Airflow	General-purpose, Python-defined, batch-oriented orchestration with big ecosystem.
Prefect / Dagster	Newer orchestrators; Dagster is asset-centric, Prefect emphasises dynamic Python flows.
dbt	SQL transformations inside a warehouse (the T in ELT). Often orchestrated BY Airflow.
Spark / pandas	Processing engines. Airflow triggers them; it does not replace them.
Azure Data Factory / AWS Glue	Cloud-managed ETL/orchestration services.

Part 3. File-by-File Deep Dive

Repository layout (everything lives under ETLWeather-main/):

```
ETLWeather-main/
  dags/etlweather.py            <- the actual pipeline (the file that matters most)
  tests/dags/test_etlweather.py  <- tests you wrote for this pipeline
  tests/dags/test_dag_example.py <- generic hygiene tests shipped by the Astro template
  Dockerfile                    <- which Airflow image to run
  airflow_settings.yaml         <- local-only connections for development
  requirements.txt              <- extra Python packages (empty)
  packages.txt                  <- extra OS packages (empty)
  .astro/config.yaml            <- Astro CLI project settings (name, Postgres port)
  README.md                     <- default Astronomer template README
  LICENSE                       <- GPL v3
```

File	Role in one line	Edited by you?
dags/etlweather.py	Defines the whole ETL DAG	Yes, core work
tests/dags/test_etlweather.py	Unit tests for this DAG (mocked API/DB)	Yes
tests/dags/test_dag_example.py	Template tests: no import errors, DAGs tagged, retries >= 2	Template (still applies to your DAG)
Dockerfile	One line: base image astro-runtime 13.12.2 (upgraded from 12.1.1)	Template, edited
airflow_settings.yaml	Seeds the 2 connections for local dev	Yes
requirements.txt	Empty (only a comment): providers already in Runtime	Template
packages.txt	Empty: no OS packages needed	Template
.astro/config.yaml	Project name etlweather; host Postgres port 5433	Partly (port)
README.md	Generic Astro boilerplate	Template (stale, see below)
LICENSE	GPL v3 text	Template/added

3.1 dags/etlweather.py - section by section

A. Imports

- `DAG`: the workflow container. `task` (decorators): TaskFlow API.
- `AirflowFailException`: raise to fail a task *immediately without retries*.
- `Param`: defines a validated runtime parameter.
- `SQLExecuteQueryOperator` (common-sql provider): runs SQL against any DB connection; the modern replacement for PostgresOperator.
- `HttpHook` (http provider) and `PostgresHook` (postgres provider): connection-aware clients for REST APIs and Postgres.

B. Constants

- `POSTGRES_CONN_ID = 'postgres_default'` and `API_CONN_ID = 'open_meteo_api'`: names of the Airflow Connections. The DAG never holds a hostname or password for them.
- `MIN_TEMPERATURE = -90, MAX_TEMPERATURE = 60`: plausibility bounds in degrees Celsius. Lowest reliably recorded air temperature on Earth is about -89 C (Antarctica) and the highest about 57 C, so the range catches sensor/API garbage without rejecting real extremes. Named constants instead of magic numbers.

C. default_args

```
default_args = {'owner': 'airflow', 'start_date': datetime(2024, 1, 1),
                'retries': 2, 'retry_delay': timedelta(minutes=5)}
```

- Applied to every task in the DAG unless a task overrides it.
- **retries=2**: 1 original attempt + 2 retries = up to 3 attempts. **retry_delay=5 min** gives transient network/API issues time to clear.
- **start_date**: first possible data interval. Passed via default_args here (fine; many projects pass it on the DAG itself).
- **owner**: label in the UI; has no security effect.

D. SQL definitions

```
CREATE TABLE IF NOT EXISTS weather_data (
    latitude FLOAT, longitude FLOAT, temperature FLOAT, windspeed FLOAT,
    winddirection FLOAT, weathercode INT,
    observed_at TIMESTAMP,                          -- when the weather was measured
    timestamp   TIMESTAMP DEFAULT CURRENT_TIMESTAMP,    -- when WE loaded the row
    UNIQUE (latitude, longitude, observed_at));
INSERT INTO weather_data (latitude, ..., observed_at) VALUES (%s, ..., %s)
ON CONFLICT (latitude, longitude, observed_at) DO NOTHING
```

- **CREATE TABLE IF NOT EXISTS**: safe to run every day; creates the table the first time only (idempotent DDL).
- **Two timestamps, two meanings:** *observed_at* = event time (when the API says the measurement was taken); *timestamp* = ingestion/load time (DEFAULT CURRENT_TIMESTAMP). Their difference is data latency. This is a classic analyst concept.
- **UNIQUE (latitude, longitude, observed_at)**: the business key. One reading per place per measurement time. Open-Meteo's current_weather refreshes in 15-minute steps, so two runs inside the same 15-minute window return the same observed_at and would otherwise insert duplicates.
- **ON CONFLICT ... DO NOTHING**: Postgres upsert syntax; if the key already exists, skip silently. Combined with the UNIQUE constraint this makes the load **idempotent**.
- **%s placeholders** with a parameters tuple: parameterised query. The driver (psycopg2) handles quoting, which prevents SQL injection and type issues. Never build SQL with f-strings from external data.
- **FLOAT** in Postgres = double precision. Fine for weather; money would need NUMERIC. **weathercode INT** stores the WMO weather code (0 clear sky, 1-3 partly cloudy, 45/48 fog, 51-67 drizzle/rain, 71-77 snow, 95-99 thunderstorm).
- Note the column literally named `timestamp`: legal in Postgres (non-reserved) but a better name would be loaded_at/ingested_at. Good self-critique to mention.

E. The DAG object

```
with DAG(dag_id='weather_etl_pipeline', default_args=default_args,
         schedule='@daily', catchup=False, tags=['weather', 'etl'],
         params={'latitude':  Param(51.5074, type='number', minimum=-90,  maximum=90),
                 'longitude': Param(-0.1278, type='number', minimum=-180, maximum=180)}) as dag:
```

- **dag_id**: unique name in the UI. **tags**: UI filtering (and required by the template test).
- **schedule='@daily'**: once per day at 00:00 UTC. (The parameter is *schedule*; older Airflow used schedule_interval.) Daily is a design choice: gives one snapshot per location per day. Hourly would give a richer time series.
- **catchup=False**: do not backfill missed intervals, because the API returns *current* weather, not historical.
- **params**: default location = London (51.5074, -0.1278). Validated with a JSON-schema: type number, latitude in [-90, 90], longitude in [-180, 180]. When someone uses "Trigger DAG w/ config" in the UI, the form shows these fields and rejects invalid values. Scheduled runs use the

defaults.

F. Task 1 - create_table (SQLExecuteQueryOperator)

Runs the CREATE TABLE IF NOT EXISTS statement through the postgres_default connection. Using an operator here (instead of a @task) is deliberate: it is the ready-made, declarative way to run SQL, with templating and connection handling built in.

G. Task 2 - extract_weather_data

- Builds an HttpHook with the open_meteo_api connection (base URL https://api.open-meteo.com) and method GET.
- Calls `/v1/forecast` with data = {latitude, longitude, current_weather: 'true'}. For GET requests the hook sends `data` as the query string.
- The `params=None` argument is how a TaskFlow function receives the DAG's Params (Airflow injects context values by argument name). `params['latitude']` is the runtime value, default or user-supplied.
- If status is 200 return response.json() (becomes an XCom); otherwise raise a generic Exception, so the task fails and is **retried** (2 times, 5 min apart). The hook also raises on HTTP errors itself; the explicit check is a defensive, readable second layer.
- Example response shape: {'latitude':..,'longitude':..,'current_weather': {'time':'2026-10-25T10:30', 'interval':900,'temperature':12.7,'windspeed':27.7,'winddirection':269,'is_day':1,'weathercode':1} }. Units: temperature C, windspeed km/h, direction degrees.

H. Task 3 - transform_weather_data

- Takes the raw JSON (an XCom from extract) and picks `current_weather`.
- Returns a flat dict with exactly the columns of the table. It adds latitude/longitude from params (the API echoes slightly adjusted coordinates, so using the request values keeps the business key stable across runs) and maps `time` to `observed_at`.
- This is a light transformation: select, rename, restructure. No heavy cleaning or unit conversion. Be honest about that in the interview, and mention what you would add (unit conversion, type casting, timezone handling, null handling).

I. Task 4 - validate_weather_data (data quality gate)

- Fails if temperature is missing (None) or outside -90..60 C.
- Raises **AirflowFailException**: marks the task failed *immediately and skips retries*. Reason: retrying will fetch the same bad data (or the failure is not transient), so retries waste time. Compare to the generic Exception in extract, where a retry can genuinely help.
- When validate fails, load becomes *upstream_failed* so nothing bad reaches the table. This pattern is called a **quality gate** or fail-fast validation.
- Returns the same dict on success, so the data flows on to load.
- Limitation: only temperature is checked. Windspeed (>= 0), winddirection (0-360), weathercode (known set) and observed_at format could be validated too.

J. Task 5 - load_weather_data

- PostgresHook(postgres_conn_id=...).run(INSERT_SQL, parameters=(...)): executes the parameterised insert and commits the transaction when the statement finishes.
- Because of ON CONFLICT DO NOTHING, running the load twice for the same reading is harmless.
- Tuple order must match column order in the INSERT. The tests assert the exact parameter tuple.

K. Workflow wiring (last four lines)

```
weather_data     = extract_weather_data()
transformed_data = transform_weather_data(weather_data)
validated_data   = validate_weather_data(transformed_data)
create_table >> load_weather_data(validated_data)
```

- Passing one task's return value into the next builds the chain extract -> transform -> validate -> load automatically.
- `create_table >> load_weather_data(...)` adds the second upstream edge for load: the table must exist before inserting. The >> operator means "runs before".
- create_table is not chained to extract, so they run in parallel: a small efficiency gain, and it shows you understand that dependencies should only be as strict as needed.

Why imports work: Operators/hooks need the matching provider packages. They are not in requirements.txt because Astro Runtime already ships apache-airflow-providers-http, -postgres and -common-sql. If you ran this on plain Airflow you would need to pip install them.

3.2 tests/dags/test_etlweather.py (your tests)

Goal: prove the pipeline logic works **without** calling the real API or database. This is done with **mocking** (unittest.mock.patch / MagicMock) to replace HttpHook.run and PostgresHook.run, and by calling the task functions directly via `dag.get_task(id).python_callable` (the raw Python function under the @task decorator).

Test	What it proves
test_dag_has_expected_tasks	The DAG parses and contains exactly the 5 expected task ids (catches accidental rename/removal).
test_task_dependencies	Wiring is right: extract->transform->validate->load, and create_table->load.
test_params_default_to_london	Defaults are 51.5074 / -0.1278.
test_params_reject_out_of_range_latitude	Param validation rejects latitude 123.
test_extract_passes_params_as_query_string	HttpHook.run is called with /v1/forecast and the right query dict (mocked response).
test_extract_raises_on_non_200	A 500 response raises an exception mentioning 500.
test_transform_builds_row_with_observed_at	Raw JSON becomes the expected flat row, with observed_at from API time.
test_validate_passes_plausible_temperature	12.7 C passes and the row is returned unchanged.
test_validate_fails_without_retry_on_bad_temperature	Parametrised: None, -91, 61 each raise AirflowFailException (3 cases).
test_load_inserts_with_conflict_guard	SQL contains ON CONFLICT ... DO NOTHING and the parameter tuple is in the correct order.

- **DagBag**: Airflow's loader that parses the dags folder. `dag_bag.import_errors` shows syntax/import problems. The fixture asserts the DAG imported, so a broken file fails the test with the error message.
- **Fixture scope=module**: the DagBag is built once and shared, which is faster.
- **Unit test vs integration test**: these are unit tests (everything external is mocked). They do not prove the real API/DB works; that is what running the DAG in Airflow demonstrates.
- Pattern used: Arrange (fake data/mocks), Act (call the function), Assert (compare to expected). Test names describe behaviour.
- Run with `astro dev pytest` (runs inside a container with Airflow installed) or pytest in an environment where Airflow is installed.

3.3 tests/dags/test_dag_example.py (template hygiene tests)

Shipped by `astro dev init`. It applies to every DAG in the folder:

- **test_file_imports**: no DAG file has import errors (DagBag.import_errors is empty).
- **test_dag_tags**: every DAG has at least one tag (your DAG has ['weather','etl']). APPROVED_TAGS is an empty set meaning "no restriction on which tags".
- **test_dag_retries**: every DAG's default_args has retries >= 2 (yours is exactly 2). This is a team policy enforced in code. Note it reads default_args, so retries must be set there to pass.
- Helpers: suppress_logging silences Airflow logs during DagBag creation; the AIRFLOW_HOME env var is used to show relative file paths.
- Why it matters: this is a basic CI safeguard so a broken or non-compliant DAG is caught before deployment.

3.4 Dockerfile

```
FROM quay.io/astronomer/astro-runtime:13.12.2
```

- A single line: use Astronomer's pre-built Airflow image (Astro Runtime), hosted on Quay. It was originally 12.1.1 (Airflow 2.10 family); Astro retired that version and rejected it when creating the cloud deployment, so it is now **13.12.2 (Airflow 2.11.2)**. Astro Runtime = open-source Airflow + tested provider packages + extras.
- **Docker** packages the app and all dependencies into an image so it runs identically anywhere. **docker-compose** (done for you by the Astro CLI) starts several containers (scheduler, webserver, triggerer, Postgres) together.
- Pinning the version (a specific number rather than latest) gives reproducible builds.
- Astro CLI copies dags/, include/, plugins/, requirements.txt, packages.txt into the image at build time. Here only dags/, tests/ and the config files exist (no include/ or plugins/ folders).

3.5 airflow_settings.yaml (local development only)

```
airflow:
  connections:
    - conn_id: open_meteo_api   conn_type: http      conn_host: https://api.open-meteo.com
    - conn_id: postgres_default conn_type: postgres  conn_host: postgres  conn_schema: postgres
      conn_login: postgres  conn_password: postgres  conn_port: 5432
```

- The Astro CLI imports this file on `astro dev start`, creating the two connections automatically, so a teammate does not need to click through the UI.
- **open_meteo_api**: an HTTP connection whose host is the API base URL. No credentials since Open-Meteo is free and keyless. The DAG supplies only the path and query.
- **postgres_default**: points to the Postgres container Astro creates. Host is **postgres** because that is the service name on the Docker network (inside Docker, localhost would mean the container itself). Port 5432 is the container's internal port; the port exposed to your laptop is 5433 (see config.yaml).
- Password postgres/postgres is the throwaway local default of the Astro dev database, and the file is labelled local-only. In production you would use a secrets backend or environment variables and never commit real credentials. Say this proactively.
- **Not used in the cloud:** a hosted Astro deployment ignores this file. There the same two connections are defined as deployment environment variables (AIRFLOW_CONN_OPEN_METEO_API, AIRFLOW_CONN_POSTGRES_DEFAULT); see Part 8.
- **Caveat to know:** this reuses Airflow's own metadata database to store weather_data. Acceptable for a learning project; in production the warehouse should be a separate database so analytics load and Airflow internals do not mix.

3.6 requirements.txt and packages.txt

- **requirements.txt**: Python (pip) packages to add on top of the image. Contains only a comment link, i.e. nothing extra. All the providers used are pre-installed in Astro Runtime.
- **packages.txt**: OS-level (apt) packages. Empty file; nothing needed (you would add e.g. libpq-dev or git if a Python package required it).
- Interview angle: knowing the difference between Python deps and system deps, and that you did not add unnecessary dependencies.

3.7 .astro/config.yaml

```
project:
  name: etlweather
postgres:
  port: "5433"
```

- **project.name**: names the docker-compose project and containers.
- **postgres.port 5433**: publish the metadata Postgres on host port 5433 instead of the default 5432. Typically done to avoid clashing with a Postgres already running on your laptop. If you connect with DBeaver/psql from your machine use localhost:5433, user postgres, password postgres, database postgres.
- The README still says 5432 because it is the generic template; your config overrides it.

3.8 README.md and LICENSE

- **README.md** is the unmodified Astronomer template. It describes the default *example_astronauts* DAG, which does not exist in your dags/ folder, and it never mentions your weather pipeline. Be ready to say so, and consider replacing it with a README describing your DAG (purpose, architecture, how to run, example queries) before the interview.
- Useful facts it does contain: `astro dev start` launches 4 containers (Postgres, Webserver, Scheduler, Triggerer); UI at http://localhost:8080 with admin/admin; Postgres at localhost:5432 by default (5433 here).
- **LICENSE**: GNU General Public License v3. Copyleft: derivative works distributed must also be GPL. Know this one-liner in case you are asked; do not claim you chose it deliberately unless you did.

3.9 How to run it locally (be able to demo or describe)

```
cd ETLWeather-main
astro dev start          # builds image, starts 4 containers, imports airflow_settings.yaml
# open http://localhost:8080  (admin / admin)
# unpause weather_etl_pipeline, then Trigger DAG (or Trigger DAG w/ config to change lat/long)
astro dev pytest         # run the tests inside the container
astro dev logs           # view component logs
astro dev stop           # stop containers
```

Tip: If the Docker/Astro CLI is not available in the interview, be able to narrate this flow and show screenshots of the Grid and Graph views.

Part 4. The Data-Analyst Angle

The interviewer is hiring an analyst, not a platform engineer. Airflow is the vehicle; the story you want to tell is "I made sure analysis-ready data arrives reliably, and I know how to question its quality."

4.1 ETL vs ELT

	ETL (this project)	ELT
Order	Extract, Transform in the pipeline, then Load	Extract, Load raw, Transform inside the warehouse
Transform where?	Python task before the DB	SQL (often dbt) inside Snowflake/BigQuery/Postgres
Good for	Small data, cleaning/validating before storage	Large data, flexibility, keeping raw history
Here	Light transform + validate, then load a clean table	Would keep the raw JSON as well and model in SQL

4.2 Data quality: what this project checks and what it could

Dimension	Meaning	Status here
Validity / plausibility	Values within allowed range	Done: temperature -90..60 C
Completeness	No missing required values	Partly: temperature None is rejected
Uniqueness	No duplicate records	Done: UNIQUE + ON CONFLICT DO NOTHING
Timeliness / freshness	Data arrives when expected	Observable via timestamp - observed_at; not alerted
Consistency	Same format across loads	Same schema, typed columns
Accuracy	Matches reality	Trusts the API; no cross-check

4.3 Event time vs ingestion time

observed_at is when the weather was measured; **timestamp** is when your pipeline stored it. Analysts should almost always chart by event time (observed_at). Ingestion time is for auditing and latency checks. If a run fails and is retried the next day, ingestion time shifts but observed_at stays true.

4.4 SQL you can run on weather_data (practise these)

```
-- 1. Latest reading per location
SELECT DISTINCT ON (latitude, longitude) *
FROM weather_data ORDER BY latitude, longitude, observed_at DESC;

-- 2. Daily temperature summary per location
SELECT latitude, longitude, observed_at::date AS day,
       ROUND(AVG(temperature)::numeric,1) AS avg_temp,
       MIN(temperature) AS min_temp, MAX(temperature) AS max_temp, COUNT(*) AS readings
FROM weather_data GROUP BY 1,2,3 ORDER BY day;

-- 3. Duplicate check (should return 0 rows thanks to UNIQUE)
SELECT latitude, longitude, observed_at, COUNT(*)
FROM weather_data GROUP BY 1,2,3 HAVING COUNT(*) > 1;

-- 4. Pipeline latency (freshness)
SELECT observed_at, timestamp AS loaded_at, timestamp - observed_at AS lag
FROM weather_data ORDER BY observed_at DESC LIMIT 10;

-- 5. Weather condition distribution
SELECT weathercode, COUNT(*) AS n,
       ROUND(100.0*COUNT(*)/SUM(COUNT(*)) OVER (),1) AS pct
FROM weather_data GROUP BY weathercode ORDER BY n DESC;

-- 6. Day-over-day change with a window function
SELECT observed_at::date AS day, temperature,
       temperature - LAG(temperature) OVER (PARTITION BY latitude, longitude
                                            ORDER BY observed_at) AS change_vs_prev
FROM weather_data;

-- 7. 7-reading moving average
SELECT observed_at, temperature,
       AVG(temperature) OVER (PARTITION BY latitude, longitude ORDER BY observed_at
                              ROWS BETWEEN 6 PRECEDING AND CURRENT ROW) AS ma7
FROM weather_data;

-- 8. Missing days (gaps): which dates have no reading?
SELECT d::date AS missing_day
FROM generate_series(CURRENT_DATE - 30, CURRENT_DATE, '1 day') d
LEFT JOIN weather_data w ON w.observed_at::date = d::date
WHERE w.observed_at IS NULL;
```

Reality check: Because the DAG runs once a day, you get about one row per location per day, so queries 2, 6 and 7 are only interesting after several days of data. Run the DAG a few times (with different lat/long) before the interview so you have real rows and screenshots to talk about.

4.5 How an analyst would use the table

- Connect Power BI / Tableau / Metabase / Looker Studio to the Postgres table and build a temperature trend line, wind rose, weather-condition bar chart, city comparison.
- Join with other datasets (sales, footfall, energy use) on date and city to study weather impact. This is the realistic business use case: demand forecasting, retail, logistics, energy.
- Use weather_data as a trusted source because the pipeline guarantees uniqueness and plausible values.
- Pandas: `pd.read_sql('select * from weather_data', engine)` for exploration.

4.6 WMO weather code quick reference (Open-Meteo)

Code	Meaning
0	Clear sky
1, 2, 3	Mainly clear, partly cloudy, overcast
45, 48	Fog, depositing rime fog
51-57	Drizzle (light to dense, incl. freezing)
61-67	Rain (slight to heavy, incl. freezing)

Code	Meaning
71-77	Snowfall, snow grains
80-82	Rain showers
85-86	Snow showers
95, 96, 99	Thunderstorm (96/99 with hail)

Part 5. Design Decisions, Limitations and Improvements

Interviewers love candidates who know their project's weak points. Own these before they are found.

5.1 Decisions you made and the reasoning

Decision	Why
Airflow instead of cron	Dependencies, retries, history, UI, parameters, alerting hooks.
TaskFlow API	Less boilerplate, automatic XCom wiring, readable Python.
Hook inside @task for HTTP, operator for SQL DDL	Custom logic needs Python (hook); simple SQL fits a ready-made operator.
Connections not hardcoded URLs/passwords	Security, portability between environments, one place to rotate.
UNIQUE + ON CONFLICT DO NOTHING	Idempotent load, safe reruns and retries.
AirflowFailException for validation	Bad data will not become good on retry; fail fast, save time and resources.
Generic Exception on HTTP failure	Network/API errors are often transient; retries can fix them.
catchup=False	API gives only current weather; backfill would mislabel data.
Params with schema validation	Change location from UI without code edits and with safe bounds.
Mocked unit tests	Fast, deterministic, no network or DB needed.
Astro CLI + Docker	Reproducible local environment that mirrors production Airflow.

5.2 Honest limitations

- **Single location per run.** To track many cities you must trigger the DAG repeatedly with different params, or extend it with dynamic task mapping.
- **Granularity is daily.** One snapshot per day does not capture the daily temperature curve. Hourly scheduling would; also Open-Meteo offers hourly/daily forecast and historical archive endpoints.
- **The time of the reading depends on when the run executes.** The DAG ignores the logical date; it always fetches "now". That is why backfills are disabled.
- **Light transform.** No unit conversion, no type checks beyond the DB, raw API JSON is not stored (no raw/bronze layer).
- **Validation covers temperature only.** No checks on windspeed, direction, weathercode or timestamp format; no row-count or freshness checks.
- **No alerting.** Failures are visible in the UI but nobody is notified (no email/Slack callback).
- **Warehouse = Airflow metadata DB.** Convenient locally, wrong for production.
- **Credentials in a YAML file.** Local-only convention; production needs a secrets backend.
- **Float equality in a UNIQUE key.** Works because the same Param value is reused as-is; rounding differences could break uniqueness if the coordinates were computed.
- **Naming:** column timestamp is ambiguous; loaded_at would be clearer. README is the template.
- **Tests are unit-level only.** No integration test against a real Postgres; no CI pipeline in the repo (no .github folder).

5.3 Improvements roadmap (pick 3 to talk about)

Improvement	How	Value
Many cities	Dynamic task mapping: extract.expand(city=list_of_cities) or a Variable holding the list	Scales to a real dataset
Hourly runs	schedule='@hourly'; use observed_at for dedupe (already supported)	Better time series
More checks	Range checks on all fields, null checks, freshness, row-count; or Great Expectations / Soda	Trustworthy data
Alerts	on_failure_callback to Slack/email, or email_on_failure	Faster response to failures
Raw layer	Store raw JSON (JSONB column or file) before transform	Reprocessing and auditing
Separate warehouse	Dedicated Postgres/BigQuery/Snowflake connection	Production-grade isolation
Secrets backend	Environment variables / Vault / cloud secrets manager	Security
Dashboard	Power BI / Tableau / Metabase over weather_data	Shows analyst value
CI/CD	GitHub Actions: lint + astro dev pytest on every push; deploy via astro deploy	Quality automation
Views / marts	SQL view for daily aggregates; dbt model	Analysis-ready layer
Historical load	Separate DAG using Open-Meteo archive API, driven by logical date	Legitimate backfill

Part 6. Interview Question Bank

Model answers are written to be spoken in 20-40 seconds. Items marked *Likely follow-up* show where interviewers usually dig next.

6.1 Project overview and motivation

Q1. Walk me through your project.

Use the pitch in Part 1: source (Open-Meteo API), 5-task DAG (create table, extract, transform, validate, load), sink (PostgreSQL), daily schedule, retries, idempotent load, quality check, tests, Docker via Astro.

Likely follow-up: *Why did you choose weather data? (Free, keyless, real JSON API, easy to validate, good for time-series analysis.)*

Q2. What problem does it solve?

It automates reliable collection of weather readings into a queryable table, with failure handling and data checks, so analysis does not depend on someone manually downloading data.

Q3. Why did you build it?

To learn orchestration and the full ETL lifecycle end to end: API ingestion, validation, SQL storage, scheduling, testing, and containerised deployment. It was my first Airflow project.

Q4. Was this a team or solo project? What was your contribution?

Answer truthfully. If solo: you designed the DAG, wrote SQL, config, and tests. If you started from the Astro template: say so, the template gave the folder structure, Dockerfile, README and the example tests, and you wrote the DAG, connections and project tests.

Q5. What is the end output of the pipeline?

Rows in the weather_data table: latitude, longitude, temperature, windspeed, winddirection, weathercode, observed_at, plus a load timestamp.

Q6. Which part are you most proud of?

Suggested: the idempotent load plus the validation gate. Together they mean reruns and bad API data cannot corrupt the table. Explain the UNIQUE key and why AirflowFailException is used.

Q7. What was the hardest problem you faced?

Pick a real one. Safe options if true: duplicate rows on reruns (solved with the UNIQUE key and ON CONFLICT); connection host confusion inside Docker (service name postgres vs localhost, ports 5432 vs 5433); understanding XCom and TaskFlow wiring.

Q8. How would you scale this to 1,000 cities?

Hold the city list in a Variable or table, use dynamic task mapping so each city is a mapped task instance, batch requests (Open-Meteo accepts multiple coordinates), use a pool to cap API concurrency, and use a distributed executor (Celery/Kubernetes). Load via batched inserts.

6.2 Airflow concepts

Q9. What is Apache Airflow?

An open-source platform to author, schedule and monitor workflows as Python code. It orchestrates tasks and their dependencies, with retries, logging and a UI.

Q10. Is Airflow an ETL tool?

Not exactly. It is an orchestrator. The ETL work is done by your Python code, SQL, or other engines; Airflow decides when and in what order they run and handles failures.

Q11. What is a DAG? Why acyclic?

A graph of tasks with directed dependencies. Acyclic means no cycles, so there is always a valid execution order and no task waits on itself forever.

Q12. Difference between DAG, task, and task instance?

DAG is the definition of the workflow; a task is one node in it; a task instance is that task in one specific DAG run with its own state, logs and try number.

Q13. What is an operator? Name some.

A template for a unit of work. Examples: BashOperator, PythonOperator, EmailOperator, SQLExecuteQueryOperator (used here), S3 operators, SimpleHttpOperator.

Q14. What is a hook and how is it different from an operator?

A hook is a client wrapper that manages the connection to an external system; you call it from code. An operator is a ready-made task, and typically uses a hook inside. I used HttpHook and PostgresHook inside @task functions where I needed custom logic.

Q15. What is the TaskFlow API?

Decorator-based style: @task turns a Python function into a task, return values travel as XComs, and calling tasks with each other's outputs defines dependencies.

Likely follow-up: *How does Airflow know extract runs before transform? (Because transform receives extract's return value, an XComArg, which creates the dependency.)*

Q16. What is XCom?

Cross-communication: a way for tasks to exchange small data via the metadata DB. In my DAG the API JSON and the transformed row are passed this way automatically through return values.

Likely follow-up: *Why not pass a large DataFrame? (Stored in the metadata DB; size limits and slowness. Write to storage and pass a path.)*

Q17. What is the scheduler?

The component that parses DAG files, creates DAG runs on schedule, evaluates dependencies, and queues ready task instances to the executor.

Q18. What does the executor do? Which one did you use?

It runs task instances. Locally the Astro CLI uses a single-machine executor (LocalExecutor). In production you might use CeleryExecutor or KubernetesExecutor to distribute work.

Q19. What is the metadata database?

Airflow's state store: DAG runs, task states, XComs, connections, variables, users. In Astro local it is a Postgres container.

Q20. What is the triggerer?

A component for deferrable operators: it waits for events asynchronously so workers are not blocked. Not used in my DAG but runs as a container in the Astro stack.

Q21. What is a Connection? Why use it?

Stored host/credential info referenced by conn_id. It keeps secrets out of code and lets the same DAG run in dev and prod with different connection definitions.

Q22. What is the difference between a Connection and a Variable?

Connection: structured credentials for external systems. Variable: general key/value configuration.

Q23. What are Params?

Runtime parameters for a DAG with optional JSON-schema validation. They appear in the Trigger DAG w/ config form. Mine are latitude and longitude with number type and min/max.

Q24. Explain schedule='@daily' and start_date.

The DAG runs once per day at midnight UTC. Each run covers a data interval and fires after the interval ends. start_date is the first interval boundary Airflow considers; it is not when the DAG first runs.

Q25. What is catchup? Why False?

catchup=True creates runs for all missed intervals since start_date. With start_date in 2024 and a daily schedule, that would create hundreds of runs, each fetching today's weather. False creates only the latest run.

Q26. What is logical date?

The start of the data interval that a run represents (formerly execution_date). It is not the actual start time of the run. My DAG ignores it because it fetches current data.

Q27. What is a backfill?

Re-running a DAG for past intervals, usually with the CLI or UI, to process historical data. Requires idempotent tasks and a source that can return historical data by date.

Q28. What does retries=2 with retry_delay=5 minutes mean?

If a task fails, Airflow retries up to 2 times, waiting 5 minutes between attempts. Total 3 attempts. After that it is marked failed.

Q29. How do task dependencies work? What does >> do?

a >> b means b runs after a succeeds (default trigger rule). Equivalent to a.set_downstream(b). Lists can express fan-out/fan-in.

Q30. What is a trigger rule?

Condition on upstream states that allows a task to run. Default all_success. Others: all_done (run regardless), one_failed (alerting tasks), none_failed, etc.

Likely follow-up: *What happens to load if validate fails? (upstream_failed; it does not run.)*

Q31. What is idempotency and why does it matter in Airflow?

Same input run many times gives the same final state. Tasks get retried and DAGs get backfilled or rerun manually, so non-idempotent tasks create duplicates or wrong totals.

Q32. How do you monitor a DAG?

Airflow UI: Grid view for run/task states, Graph view for structure, per-task logs, duration charts, plus alerts (callbacks, email). Task logs are the first place to debug.

Q33. How would you debug a failed task?

Open the task instance logs in the UI, read the traceback, check the connection settings and upstream XCom values, reproduce by running the function locally or with astro dev run / pytest, fix, then Clear the task to rerun.

Q34. How do you rerun a failed task?

Clear the task instance in the UI (optionally with downstream) so the scheduler runs it again; or trigger a new DAG run.

Q35. What is a sensor?

A task type that waits for a condition (file arrival, row existence, external task). Modes: poke, reschedule, deferrable.

Q36. What is dynamic task mapping?

A feature to create task instances at runtime from data, using .expand(). For example, one extract task per city from a list.

Q37. Why Astro CLI? What does it give you?

A one-command local Airflow stack in Docker (scheduler, webserver, triggerer, Postgres), project scaffolding, config import, and test/deploy commands. It removes the pain of installing Airflow manually.

Q38. What is Docker and why is it used here?

Containers package an app with its dependencies so it behaves identically on any machine. Here the Astro Runtime image bundles Airflow, providers and OS libraries so my pipeline runs the same everywhere.

Q39. Which Airflow version and does it matter?

Locally it started on Astro Runtime 12.1.1 (Airflow 2.10 family). Astro had retired that version, so the cloud deployment and the Dockerfile use Runtime 13.12.2, which is Airflow 2.11.2. The decorators import path airflow.decorators and the schedule= parameter are 2.x style; Airflow 3 moves the authoring API to airflow.sdk but keeps compatibility imports for a while. I stayed on 2.x so the DAG needed no changes.

6.3 The code (file-level, concept-level)

Q40. Why does create_table run before load? Why not before extract?

load needs the table to exist. Extract does not depend on the table, so it can run in parallel, which saves time.

Q41. Why use CREATE TABLE IF NOT EXISTS?

So the task is safe to run every day. It creates the table the first time and does nothing afterwards (idempotent DDL).

Q42. Why separate extract, transform, validate and load into tasks?

Separation gives clear failure points, retries at the right granularity (retry just the API call), visibility in the UI, and easier tests. The trade-off is XCom overhead, acceptable for tiny payloads.

Q43. How does extract get the latitude and longitude?

From the DAG's Params, injected into the task function as the params argument. Defaults to London; can be overridden when triggering manually.

Q44. What URL does extract actually call?

GET https://api.open-meteo.com/v1/forecast?latitude=..&longitude=..&current_weather=true. The base comes from the open_meteo_api connection; the path and query are in the task.

Q45. What does the API return?

JSON with metadata and a current_weather object: time, interval (900 s), temperature (C), windspeed (km/h), winddirection (degrees), is_day, weathercode.

Q46. What happens if the API is down?

extract raises, the task goes up_for_retry, retried twice at 5-minute intervals. If it still fails, extract is failed and downstream tasks are upstream_failed. Nothing is written.

Likely follow-up: *Does create_table still run? (Yes; it is independent.)*

Q47. What does transform do?

Extracts current_weather from the response, adds the request coordinates, renames time to observed_at, and returns a flat dict that matches the table columns.

Likely follow-up: *Is that really a transformation? (Light one. I would add unit handling and type casting.)*

Q48. Why take latitude and longitude from params and not from the API response?

The API snaps coordinates to its model grid, so the returned values can differ slightly from what you asked for. Using the requested values keeps the business key stable for the UNIQUE constraint.

Q49. Explain your data-quality check.

Rejects missing temperature or values outside -90 to 60 C. Those bounds cover real Earth extremes (about -89 and 57 C record) but catch corrupted values like -999 or 9999.

Q50. Why AirflowFailException instead of a normal Exception?

A normal exception triggers retries. If the data is bad, retrying fetches the same bad data, wasting 10 minutes. AirflowFailException fails immediately with no retry.

Q51. Why validate before load, not after?

Prevent bad data from ever entering the table (gate). Cleaning it later requires deletes and may already have fed a dashboard.

Q52. Explain the INSERT statement.

Parameterised INSERT of 7 columns. ON CONFLICT on the unique key DO NOTHING, so a repeated reading is skipped instead of raising an error or duplicating.

Q53. Why parameterised queries (%s)?

Prevents SQL injection and handles types and quoting safely. Never concatenate values into SQL.

Q54. What does UNIQUE (latitude, longitude, observed_at) enforce?

At most one row per location per measurement time. It is a composite natural key.

Likely follow-up: *Why not a surrogate id? (Could add SERIAL id too; the natural key is what dedupes. An id is optional.)*

Q55. What is the difference between observed_at and timestamp?

observed_at: when the weather was measured (from API, event time). timestamp: when my pipeline inserted it (default CURRENT_TIMESTAMP, ingestion time).

Q56. What if the DAG runs twice in a day?

Open-Meteo updates every 15 minutes. If the second run falls in a different 15-min slot, a new row is added; if in the same slot, the conflict guard skips it. Either way no duplicates.

Q57. Which timezone is observed_at in?

Open-Meteo returns times in GMT/UTC unless a timezone parameter is passed, and I did not pass one. Verify against a real response and state it as UTC. TIMESTAMP in Postgres has no timezone; TIMESTAMPTZ would be more explicit.

Q58. Why FLOAT and not DECIMAL?

Weather measurements are approximate; floating point is fine. DECIMAL/NUMERIC is for exact values like money.

Q59. Why is the Postgres host 'postgres' in airflow_settings.yaml?

Inside the Docker network, containers reach each other by service name. localhost would refer to the Airflow container itself.

Q60. Why port 5433 in .astro/config.yaml but 5432 in the connection?

5433 is the port published on my laptop (to avoid clashing with a local Postgres). Inside the Docker network Postgres still listens on 5432, which is what the Airflow containers use.

Q61. Is it a problem that the data is in Airflow's metadata database?

Fine for local learning. In production keep analytics data in a separate warehouse so Airflow stays lightweight and permissions are separate.

Q62. Where are the credentials? Is that safe?

In airflow_settings.yaml, which the README describes as local-only; the values are the throwaway defaults of the dev DB and the API needs no key. In production use environment variables or a secrets backend and never commit real secrets.

Q63. What does requirements.txt contain? Why empty?

Only a comment. The needed providers (http, postgres, common-sql) are already included in Astro Runtime, so nothing extra needs installing.

Q64. What is packages.txt?

OS-level packages installed in the image via apt. Empty because I need none.

Q65. What does the Dockerfile do?

Selects the base image quay.io/astronomer/astro-runtime:13.12.2 (I upgraded it from 12.1.1 when Astro retired that version). The Astro CLI layers my DAGs and requirements on top.

Q66. Explain why TaskFlow tasks have a params=None argument.

Airflow passes context values (like params, ti, logical_date) to a task function if the argument name matches. Default None keeps the function callable directly in tests.

Q67. Why not use the PostgresOperator for the insert?

I needed the transformed data from XCom as parameters. Doing it in a @task with PostgresHook was simpler than templating XCom into an operator. A SQLExecuteQueryOperator with parameters from XCom would also work.

6.4 Testing

Q68. How did you test the pipeline?

pytest with mocks. I load the DAG through DagBag, check structure and dependencies, call each task function directly with sample data, and patch HttpHook.run and PostgresHook.run to avoid network and DB use.

Q69. What is mocking and why?

Replacing a real dependency with a controllable fake. It makes tests fast, deterministic and independent of internet and database.

Q70. What does DagBag do?

Parses DAG files and collects DAG objects and import errors. Tests use it to verify the DAG imports and to inspect tasks.

Q71. What does test_dag_example.py check?

That no DAG has import errors, every DAG has tags, and default_args retries >= 2. They are generic hygiene checks from the Astro template.

Q72. What do your tests not cover?

Real API and real Postgres behaviour (integration), scheduler behaviour, and performance. I verified those by running the DAG in the local Airflow UI.

Q73. How would you add CI?

GitHub Actions workflow on push: install Astro CLI or Airflow, run ruff/flake8 and astro dev pytest (or pytest). Block merges if failing.

Q74. How do you test that retries are not used for bad data?

The test asserts AirflowFailException is raised for None, -91, 61. That exception type is what tells Airflow not to retry.

6.5 Data and SQL

Q75. What is ETL?

Extract data from a source, Transform it (clean, reshape, validate), Load it to a destination. My project: API, light transform plus validate, Postgres.

Q76. ETL vs ELT?

ETL transforms before loading; ELT loads raw first then transforms inside the warehouse with SQL/dbt. ELT is common with modern cloud warehouses.

Q77. What is a data pipeline?

An automated flow that moves and prepares data from sources to destinations, with scheduling, monitoring and error handling.

Q78. What is data quality? How did you ensure it?

Fitness of data for use: validity, completeness, uniqueness, timeliness, consistency, accuracy. I enforced range validity, null check on temperature, and uniqueness via a constraint.

Q79. What is a primary key vs unique constraint?

Primary key: unique and not null, one per table, row identity. Unique constraint: values must be distinct, can be several per table, allows NULLs. Mine is a composite unique constraint without a primary key.

Likely follow-up: *Should the table have a primary key? (Probably yes: a surrogate id or the composite as PK, and observed_at NOT NULL.)*

Q80. What does ON CONFLICT DO NOTHING do vs DO UPDATE?

DO NOTHING skips the conflicting row; DO UPDATE (upsert) updates existing row with new values. For immutable readings, DO NOTHING is right.

Q81. Write a query for average daily temperature per city.

SELECT latitude, longitude, observed_at::date AS day, AVG(temperature) FROM weather_data GROUP BY 1,2,3 ORDER BY 3; (see Part 4.4 for more).

Q82. How would you find duplicates?

GROUP BY the business key HAVING COUNT(*) > 1. In my table the UNIQUE constraint should make it return nothing.

Q83. What is a window function? Give an example.

A function computed across related rows without collapsing them, e.g. LAG(temperature) OVER (PARTITION BY location ORDER BY observed_at) for change vs previous reading, or a moving average.

Q84. How would you handle missing values in analysis?

Understand why they are missing; drop, impute (mean/median/forward fill) or flag, depending on the use. In the pipeline I reject rows with missing temperature rather than impute.

Q85. What is an outlier and how does your pipeline treat it?

A value far from the rest. My pipeline rejects physically impossible values via fixed bounds. Statistical outliers (e.g. unusually hot but possible) are kept; analysts handle them with z-score/IQR.

Q86. How would you visualise this data?

Line chart of temperature over time per city, bar chart of weather code frequency, wind direction rose, KPI cards for latest temp, filters by location and date.

Q87. What insights could this data provide?

Temperature trends and seasonality, extremes, wind patterns, frequency of rain/fog, comparisons between cities, and correlation with business metrics such as sales or energy demand.

Q88. What is a time series and what should you watch for?

Observations ordered by time. Watch for gaps, irregular intervals, seasonality, trend, time zones, and duplicate timestamps. Daily sampling here is coarse for weather.

Q89. What is normalisation? Is your table normalised?

Organising data to reduce redundancy. My table is a single fact-style table with repeated lat/long per row. With many cities I would add a locations dimension table (city id, name, lat, lon) and reference it.

6.6 Failure scenarios (cross-examination favourites)

Scenario	What happens	Why it is OK / how to improve
API returns HTTP 500	extract raises, retried twice, then failed; transform/validate/load upstream_failed	No bad data; add alert
API returns temperature 999	validate raises AirflowFailException, no retries; load skipped	Bad data blocked; log shows value
API changes JSON structure	transform KeyError, retried (wastefully), then failed	Add schema check with AirflowFailException
Postgres down at load	load fails and retries; validated row lives in XCom so earlier steps are not repeated if only load is cleared	Good isolation
DAG triggered twice quickly	Second insert hits UNIQUE conflict and is skipped	Idempotent
Invalid latitude 123 on trigger	Param validation rejects the trigger	Fail early at input
Scheduler down for 3 days	catchup=False: only latest interval runs on recovery; 2 days of readings missing	Acceptable for current-weather; historical API would fill gaps
Table dropped manually	create_table recreates it next run	Self-healing DDL
Task code has a syntax error	DAG fails to import; shown as an import error in UI; test_file_imports catches it	CI would prevent deploy

6.7 Wider data-analyst questions that may follow

Q90. Why Python and SQL for this?

Python for API calls and orchestration glue; SQL for storage and analysis. They are the two core analyst tools.

Q91. How do you ensure the data you analyse is trustworthy?

Know the lineage (source, transformations), check duplicates/nulls/ranges/freshness, reconcile counts, document assumptions, and automate checks like my validation task.

Q92. What is data lineage?

The traceable path of data from source through transformations to output. Here: Open-Meteo API -> XCom dicts -> weather_data -> dashboards. Airflow's DAG graph documents part of it.

Q93. What is a data warehouse vs a database?

Operational DB (OLTP) serves applications with many small transactions; a warehouse (OLAP) is optimised for analytical queries over large history. My Postgres table is a small analytical store.

Q94. What is batch vs streaming?

Batch processes data in scheduled chunks (my daily run); streaming processes events continuously (Kafka, Flink). Airflow is batch-oriented.

Q95. What is an API and what is REST?

An interface for programs to talk. REST uses HTTP verbs on URLs and typically returns JSON. I used a GET request with query parameters.

Q96. What is JSON? How do you parse it in Python?

A text format of nested key/value pairs and arrays. response.json() returns a dict I index like data['current_weather']['temperature'].

Q97. What are rate limits and how do they matter?

APIs cap request counts. One call per day is far below the limit; at scale you would add pools, batching and backoff. Open-Meteo's free tier is for non-commercial use.

Q98. What would you do if stakeholders want a dashboard?

Connect BI tool to Postgres (read-only user), build a daily aggregate view, define KPIs with the stakeholder, set refresh after the DAG finishes, document metric definitions.

Part 7. Behavioural Answers, Traps and Cheat Sheets

7.1 STAR-style answers (fill the blanks with your real experience)

Prompt	Skeleton
Tell me about a challenge	Situation: first Airflow project, pipeline produced duplicates / connection failed in Docker. Task: make reruns safe / connect services. Action: added UNIQUE + ON CONFLICT / learned Docker service names and ports. Result: reruns are harmless / pipeline runs end to end.
How did you learn Airflow?	Official docs and Astronomer tutorials, built the DAG iteratively, wrote tests, read task logs when things failed.
What would you do differently?	Add alerting, more validations, a raw layer, a separate warehouse, CI, dashboard; write a proper README.
Tell me about a time you ensured quality	Added the validation gate and tests; chose fail-fast semantics deliberately.
How do you handle ambiguity?	Started with a minimal DAG on one location, then added params, validation, idempotency step by step.

7.2 Traps: things not to say

- Do not say Airflow processes the data or "is an ETL tool" without nuance. It orchestrates.
- Do not claim production deployment, large data volumes, or users unless true. This ran locally in Docker.
- Do not claim a dashboard, row counts or uptime you have not verified.
- Do not say the pipeline gets historical weather. It fetches the *current* reading each run.
- Do not say the DAG is real-time. It is daily batch.
- Do not say retries fix bad data. They fix transient failures only.
- Do not describe XCom as a place for big data.
- If you started from the Astro template, say so confidently; use of scaffolding is normal.
- If asked something you do not know: say how you would find out (docs, logs, a quick experiment). That is a good answer.

7.3 One-page cheat sheet

Topic	Remember
Pipeline	create_table || extract -> transform -> validate -> load (load also after create_table)
Schedule	@daily, catchup=False, start 2024-01-01, UTC
Reliability	retries=2, delay 5 min, AirflowFailException for bad data, idempotent load
Config	Connections open_meteo_api, postgres_default; Params latitude/longitude (default London)
Data	weather_data: lat, lon, temperature, windspeed, winddirection, weathercode, observed_at, timestamp
Key	UNIQUE (latitude, longitude, observed_at); ON CONFLICT DO NOTHING
Quality	temperature not None and in [-90, 60] C
Env	Astro Runtime 13.12.2 (Airflow 2.11.2), Docker, astro dev start, UI :8080 admin/admin, local Postgres host port 5433
Cloud	Astro deployment etlweather, AWS us-east-1, dev mode; connections as AIRFLOW_CONN_* env vars; Supabase Postgres via IPv4 Session pooler; DAG-only deploy

Topic	Remember
Tests	pytest, DagBag, mock HttpHook.run and PostgresHook.run; 15 passed = 12 pipeline + 3 template
Airflow parts	Scheduler, executor, webserver, triggerer, metadata DB, DAG folder
Key words	DAG, task, operator, hook, connection, XCom, TaskFlow, params, catchup, idempotency, retries, trigger rule

7.4 Glossary

Term	Definition
Backfill	Running a DAG for past intervals
Business key	Natural identifier of a record (here lat, lon, observed_at)
DAG run	One execution of a DAG
Data interval	Time window a run covers
Deferrable	Task type that frees its worker slot while waiting
Event time	When something happened (observed_at)
Ingestion time	When data was stored (timestamp)
Idempotent	Safe to repeat with the same result
Orchestration	Coordinating when and in what order jobs run
Provider	Package of integrations (operators/hooks) for external systems
Quality gate	Check that blocks bad data from moving forward
Upsert	Insert or update on conflict
WMO code	Standard numeric code for weather condition

7.5 Last-day checklist

- Run `astro dev start`, trigger the DAG 2-3 times (once with a different lat/long, e.g. New York 40.71, -74.01), open Grid/Graph views and one task log.
- Query the table; note the real row count, an example row, and run 2 or 3 of the Part 4.4 queries. Take screenshots.
- Trigger with latitude 123 to see the validation error message.
- Run `astro dev pytest` and note the actual number of passing tests.
- Practise drawing the DAG diagram and the architecture diagram from memory.
- Rehearse the 30-second and 2-minute pitches aloud; time them.
- Rewrite README.md to describe your project (optional but impressive).
- Prepare 3 improvements and 3 limitations you can discuss without hesitation.
- Prepare 2 questions to ask the interviewer (their data stack, how analysts use pipelines, data quality practices).

You do not need to know all of Airflow. A fresher who clearly explains one pipeline they built, why each choice was made, how it fails, and how it could improve will beat someone who recites definitions. Good luck.

Part 8. Deploying to Astronomer (Astro Cloud)

After running the pipeline locally, the project was deployed to a hosted Airflow on Astro (Astronomer's managed service), with Supabase PostgreSQL as the destination database. This part explains what was deployed, how it differs from the local setup, the problems that came up and how they were solved, and how to talk about it. Expect questions here: deployment shows you can take a project beyond a laptop.

Be accurate: This ran as a personal learning deployment (dev mode, small scheduler), not a production system. Say that plainly.

8.1 What is deployed

Item	Value
Platform	Astro (Astronomer) hosted, standard deployment, workspace "Data Analytics"
Deployment name	etlweather
Cloud / region	AWS, us-east-1
Runtime	Astro Runtime 13.12.2, based on Airflow 2.11.2
Mode / size	Development mode (hibernates when idle, no uptime SLA), small scheduler
Executor	Left at the CLI default (CeleryExecutor per the CLI help); verify in the deployment settings
Code delivery	DAG-only deploy: `astro deploy <deployment-id> --dags`
API connection	Env var AIRFLOW_CONN_OPEN_METEO_API (JSON: conn_type http, host https://api.open-meteo.com)
DB connection	Secret env var AIRFLOW_CONN_POSTGRES_DEFAULT pointing at Supabase Postgres via the Session pooler
Destination	Supabase PostgreSQL 17, table weather_data created by the DAG's create_table task

8.2 Local vs cloud: what changes

Aspect	Local (astro dev)	Cloud (Astro deployment)
Where it runs	Docker containers on the laptop	Managed infrastructure on AWS
Connections	airflow_settings.yaml, imported at startup	Environment variables named AIRFLOW_CONN_<CONN_ID>; the yaml file is NOT used
Database	Postgres container (host name postgres)	External Supabase Postgres via the pooler
Secrets	Plain values in a local-only file	Secret env vars, masked in the UI/CLI (shown as ****)
Runtime version	12.1.1 originally	13.12.2 (12.1.1 was retired and rejected by Astro)
Access	localhost:8080, admin/admin	Authenticated Astro URL, workspace permissions
DAG new state	Created unpaused in the local setup	A new DAG starts paused; unpause then trigger
Cost	Free	Billed by usage/credits; delete when done

Key idea: Airflow reads any env var named AIRFLOW_CONN_<ID> as the connection with that conn_id (id in lower case). So AIRFLOW_CONN_POSTGRES_DEFAULT becomes the connection postgres_default that the DAG already uses. The DAG code did not change at all between local and cloud. That is the payoff of using Connections instead of hardcoded values.

8.3 Steps that were followed

```
astro login --token-login <token>      # browserless login (token from cloud.astronomer.io/token)
astro deployment create -n etlweather -p aws --region us-east-1 \
      -v 13.12.2 -m enable --scheduler-size small -f --wait
astro deployment variable create -d <deployment-id> 'AIRFLOW_CONN_OPEN_METEO_API=...'
astro deployment variable create -d <deployment-id> --secret 'AIRFLOW_CONN_POSTGRES_DEFAULT=...'
astro deploy <deployment-id> --dags -f # upload dags/ only (no image build)
astro deployment logs <deployment-id> --scheduler -k weather_etl_pipeline
astro logout                           # when finished
```

- Dockerfile base image was changed from astro-runtime:12.1.1 to 13.12.2 so the project matches the deployment runtime.
- After the deploy the DAG appeared in the Airflow UI; it was unpaused and triggered there. Task and row checks followed (see 8.5).

8.4 Problems hit and how they were solved (great STAR material)

Problem	Cause	Fix
astro login failed: address already in use (port 12345)	Browser login uses a local callback port; something else held it	Used token login (--token-login) which needs no local port
Deployment create: "12.1.1 is not a valid astro runtime version"	Astro retires old Runtime versions	Looked up current versions, chose 13.12.2 (latest Airflow 2.x line) and updated the Dockerfile; DAG code unchanged
Image build failed: TLS handshake timeout to quay.io	Docker could not reach the image registry from this network	Used a DAG-only deploy, which uploads dags/ and reuses the runtime image already in the deployment
Supabase direct host unreachable	db.<ref>.supabase.co has only an IPv6 address; the local container and likely Astro workers are IPv4-only	Switched to the Supabase Session pooler host (IPv4), username postgres.<project-ref>
Connection URI rejected / wrong password	Special character % in the password must be URL-encoded as %25 in a connection URI	Encoded the password in the URI
Secret was pasted into a chat/terminal	Credential exposure	Rotate the password and update the secret env var; never commit secrets; use --secret so values are masked

8.5 How the deployment was verified

- **Unit tests** in the Astro Runtime 13.12.2 container: 15 passed (12 for the weather DAG, 3 template hygiene tests).
- **DAG parse in the cloud:** scheduler logs show the DAG retrieved from the dags folder with no import error.
- **Connection test:** logged into the Supabase pooler from a container with psycopg2; Postgres 17 answered.
- **SQL test on Supabase:** ran the DAG's own CREATE TABLE and INSERT. First insert wrote 1 row, the identical second insert wrote 0 rows (ON CONFLICT DO NOTHING), then the transaction was rolled back so no test data remained.

- **End-to-end cloud run:** a manual trigger plus a scheduled run both executed. Supabase then contained one London reading (observed_at 11:30 UTC, about 13 C) loaded at 11:31. Both runs fetched the same 15-minute reading, so only one row exists: deduplication working in practice.
- **For the interview:** open the Grid view, make sure all 5 tasks are green, and take a screenshot plus a screenshot of the Supabase table.

8.6 Concepts you should be able to explain

Term	Explanation
Astro / Astronomer	Company behind a managed Airflow service. Astro Runtime = tested Airflow distribution; Astro CLI = tooling for local dev and deploy.
Workspace / Deployment	Workspace groups deployments and people. A Deployment is one hosted Airflow environment (scheduler, workers, metadata DB, UI).
Image deploy vs DAG-only deploy	Image deploy rebuilds the Docker image (needed when Dockerfile, requirements or packages change). DAG-only deploy uploads just the dags folder, which is faster and needs no Docker build.
Development mode	Cost-saving deployment type that can hibernate; no uptime guarantees. Not for production.
Deployment environment variables	Key/values injected into every Airflow component. Used here to define connections without the UI. Can be marked secret.
Connection URI	postgresql://user:password@host:port/database. Special characters in the password must be percent-encoded.
Supabase and pooler	Supabase is hosted Postgres. The direct database host is IPv6-only; the Supavisor pooler offers IPv4. Session mode (port 5432) behaves like a normal connection; transaction mode (6543) suits serverless clients.
IPv4 vs IPv6	Two IP address families. A client with only IPv4 cannot reach an IPv6-only host, hence the failed lookup/unreachable errors.
CI/CD for DAGs	Automate test then deploy on every push (for example GitHub Actions running pytest then astro deploy with an API token). Not set up here; it is the natural next step.
Secrets backend	Dedicated store (Vault, AWS Secrets Manager) that Airflow reads connections from, safer than env vars at scale.
Hibernation	Dev-mode deployments scale down when idle and wake on demand; the first run after sleep can be slower.

8.7 Security notes to mention proactively

- Credentials were set as **secret** deployment variables, so they are masked after saving. Nothing sensitive is committed in the DAG code or repository; airflow_settings.yaml holds only local throwaway values.
- Any credential that was ever pasted into a chat, terminal log or screenshot should be treated as exposed: **rotate it** and update the variable. Say you know this practice.
- The Astro login token is short-lived (it expired in about an hour) and should be revoked or logged out after use (astro logout).
- Least privilege: for production use a dedicated database role that can only write to the needed schema, not the admin user.
- Network: allowed IP ranges and private connectivity can restrict who reaches the deployment and the database.

8.8 Deployment-related interview questions

Q99. Did you deploy the project? Where?

Yes. I deployed it to an Astro (Astronomer) hosted Airflow deployment on AWS us-east-1, in dev mode with a small scheduler, running Airflow 2.11.2. The destination table lives in Supabase PostgreSQL. It was a learning deployment, not production.

Q100. How did you get your DAG into the cloud?

With the Astro CLI: authenticate, create a deployment, set connections as environment variables, then `astro deploy --dags`, which uploads only the dags folder. After parsing, I unpaused and triggered it in the Airflow UI.

Q101. What changed in your code between local and cloud?

No DAG logic changed. Only environment configuration: connections moved from airflow_settings.yaml to deployment env vars, the database host moved to Supabase, and the Dockerfile runtime version was bumped from 12.1.1 to 13.12.2 because 12.1.1 was retired.

Q102. Why did the runtime version change matter?

Astro only accepts supported Runtime versions. 12.1.1 was rejected, so I picked 13.12.2, the latest Airflow 2.x runtime. Staying on 2.x avoided changing import paths that differ in Airflow 3. I re-ran all 15 tests on it and they passed.

Q103. How did you handle credentials in the cloud?

As secret deployment environment variables named AIRFLOW_CONN_POSTGRES_DEFAULT and AIRFLOW_CONN_OPEN_METEO_API. Airflow maps them to the connection ids used in the DAG, so no secret is in code. In production I would use a secrets backend and a least-privilege DB user.

Q104. What is the difference between airflow_settings.yaml and environment variables?

The yaml is a local-development convenience read by the Astro CLI; it is not used by a hosted deployment. Environment variables are the deployment-level way to define connections and variables.

Q105. What went wrong during deployment and how did you fix it?

Several things: the browser login port was busy so I used token login; the old runtime version was retired so I upgraded; Docker could not reach quay.io so I used a DAG-only deploy; Supabase's direct host was IPv6-only so I used the IPv4 session pooler with username postgres.<ref>; and a % in the password needed URL-encoding as %25.

Q106. Why was the Supabase connection failing initially?

DNS for the direct host returned only an IPv6 address. My environment and the container network had no IPv6 route, so the connection was unreachable. The Session pooler exposes IPv4.

Q107. What is the difference between session and transaction pooler modes?

Session mode keeps a server connection for the whole client session (supports prepared statements, SET, etc.). Transaction mode returns the connection after each transaction, giving higher concurrency but limited features. Airflow tasks use short connections, so session mode is simple and safe.

Q108. Why a DAG-only deploy?

Only the DAG file changed and the deployment already ran a suitable runtime image. A DAG-only deploy skips the Docker build, which was also failing because the registry was unreachable.

Likely follow-up: *When would you need a full image deploy? (When requirements.txt, packages.txt, Dockerfile or include/plugins change.)*

Q109. How did you verify the cloud pipeline works?

Checked the scheduler logs for the DAG parse and queued tasks, then queried the Supabase table and found the loaded London reading with observed_at and load timestamps. I also confirmed idempotency: two runs, one row.

Q110. What is dev mode / hibernation?

A cheaper deployment type for non-production use: it can hibernate when idle and carries no uptime SLA. Fine for learning, not for critical pipelines.

Q111. What does a new DAG's paused state mean, and why is it the default?

Paused DAGs are not scheduled. New DAGs start paused so unfinished or misconfigured code does not run automatically. You unpause to enable scheduling. That also meant my first cloud run was a deliberate trigger.

Q112. How would you automate deployment?

A GitHub Actions workflow: on push run lint and pytest, then run astro deploy using a deployment API token stored as a repository secret. Also enable CI/CD enforcement so only the pipeline can deploy.

Q113. How would you promote from dev to production?

Create a separate production deployment (not dev mode, with HA if needed), separate secrets and a production database, deploy the same tested code from CI after review, and monitor with alerts.

Q114. What does the deployment cost and how did you control it?

Billing is usage-based. I used the smallest scheduler in dev mode so it can hibernate, and I plan to delete the deployment after demonstrating it.

Q115. Why Supabase and not the Astro metadata database?

Airflow's metadata DB is managed by Astro and not for application data; separating analytics data from orchestrator state is good practice. Supabase gives a free, externally reachable Postgres.

Q116. What would you monitor in production?

Task failures and retries, DAG run duration, data freshness (gap between observed_at and load time), row counts per day, and API error rates, with alerts via callbacks to Slack or email.

8.9 Cleanup and cost control

```
astro deployment list                       # confirm what exists
astro deployment delete <deployment-id>     # remove the hosted deployment when finished
astro logout                                # discard the CLI token
```

- Rotate the database password if it was ever exposed, then update the secret variable if the deployment is kept.
- Keep screenshots of the Airflow UI, successful run, and the Supabase table as interview evidence before deleting anything.